

# Model Evaluation - VisionGATE Solutions

GATE Data Science and Artificial Intelligence (GATE DA)

## Topics Index

Click on any sub-topic below to navigate directly to it:

1. [Bias-Variance](#)
2. [Validation](#)
3. [Regularization](#)

# 1 Bias–Variance

## Key ideas for this section

1. **The decomposition, and what each term averages over.** For a fixed test point  $x$ , with  $y = f(x) + \varepsilon$ ,  $\mathbb{E}[\varepsilon] = 0$ ,  $\text{Var}(\varepsilon) = \sigma^2$ , and  $\hat{f}_{\mathcal{D}}$  the model fitted to a random training set  $\mathcal{D}$ ,

$$\mathbb{E}_{\mathcal{D}, \varepsilon} \left[ (y - \hat{f}_{\mathcal{D}}(x))^2 \right] = \underbrace{\left( \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - f(x) \right)^2}_{\text{Bias}^2} + \underbrace{\text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

The expectation that defines bias and variance is over the *random draw of the training set*. This is the single fact that most of the conceptual questions in this section test.

2. **Bias and variance describe an algorithm, not a fitted model.** You cannot look at one trained model and read off its variance; variance is the spread you would see across the many models you would have obtained from other training samples drawn from the same distribution. This is why the dartboard picture needs many darts.
3. **The capacity dial.** Increasing capacity moves bias down and variance up. Training error falls monotonically; test error is U-shaped, and its minimum sits where the marginal gain in bias exactly cancels the marginal loss in variance. No amount of tuning pushes expected test error below  $\sigma^2$ .
4. **Diagnosis from two numbers.** Given a training error, a validation error and an estimate of the noise floor (or Bayes error),

avoidable bias  $\approx$  train – noise floor,      variance gap  $\approx$  validation – train

Large first quantity and small second: underfitting. Small first and large second: overfitting. Both large: both problems at once.

5. **Which lever moves which term.** More data reduces variance and leaves bias essentially untouched. More capacity, more features, or a weaker penalty reduce bias and raise variance. Stronger regularization, fewer features, bagging and early stopping reduce variance at the cost of bias. Nothing on this list touches  $\sigma^2$  — reducing that requires better measurements or genuinely new information, i.e. changing the problem rather than the model.

**Q1.** Irreducible error arises from:

**Answer: (C)**

**Solution.** Each distractor names a real quantity; the task is to match the name to the right term of the decomposition.

**(A) too simple a hypothesis class — this is bias.** If the truth is quadratic and you only ever fit straight lines, then *no* training set produces the right answer, so the average prediction is systematically wrong. That is  $\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - f(x)\right)^2$ , not  $\sigma^2$ .

**(B) sensitivity to the particular training sample — this is variance.** Refit on a different sample, get a different model. That is  $\text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x))$ .

**(C) inherent noise in the target — correct.** Write  $y = f(x) + \varepsilon$ . Even the oracle predictor  $\hat{f} = f$  incurs

$$\mathbb{E}[(y - f(x))^2] = \mathbb{E}[\varepsilon^2] = \sigma^2$$

The two patients with identical charts who have different outcomes, the two houses with identical listings that sell for different prices — that variation is not a function of  $x$ , so no function of  $x$  can predict it.

**(D) numerical errors in the optimizer.** Real, but not a term in this decomposition. The decomposition is a statement about the statistical estimator, derived assuming the optimization is solved exactly.

***Note.** The word “irreducible” is scoped to a fixed feature set. If you go out and measure a new covariate that genuinely explains part of the previously unexplained variation in  $y$ , then  $\sigma^2$  for the new problem is smaller — but that is a different problem with a different  $x$ . This distinction is exactly what Q18(C) turns on, so it is worth fixing now.*

**Q2.** The bias of a learning algorithm at a point  $x$  is:

**Answer: (B)**

**Solution.** By definition,

$$\text{Bias}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - f(x)$$

Read it in two steps: first average the prediction at  $x$  over all training sets you might have drawn, then compare that average to the truth.

**Why (A) fails.** The error of the one model you happened to train mixes bias, variance and noise together. A high-variance, zero-bias algorithm can produce a single model with enormous error; that error is not bias.

**Why (C) fails.** The spread of predictions across training sets is precisely the definition of variance. Bias is about where the centre sits, variance about how wide the cloud is; (B) and (C) are the two halves of the same picture.

**Why (D) fails.** Validation error minus training error is the empirical generalization gap. It is a useful *diagnostic* for variance (see Q14–Q16),

not a definition of bias — and it is computed from one training run, whereas bias is an average over training runs.

***Note.** Notice that  $f(x)$  appears in the definition. Bias is unobservable in practice: you would need to know the true regression function to compute it. Everything you actually measure — training error, validation error, the gap between them — is a proxy. Keeping the definitions and the diagnostics separate in your head prevents a whole family of errors.*

**Q3.** Select all true statements.

**Answer:** (A, D, E)

**Solution.** (A) **True.**  $\text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x))$  is taken at a fixed  $x$ , over the randomness in  $\mathcal{D}$ . The test point is held fixed; it is the training set that varies.

(B) **False.**  $\sigma^2 = \text{Var}(\varepsilon)$  is a property of the data-generating distribution. Collecting more samples estimates that distribution better, but does not change it. More data shrinks variance, not  $\sigma^2$ .

(C) **False.** Counterexample: the predictor  $\hat{f}(x) = 42$  for all  $x$ , regardless of the training data. It does not depend on  $\mathcal{D}$  at all, so its variance is exactly zero, while its bias is  $42 - f(x)$ , which is generally enormous. Zero variance and zero bias are independent conditions.

(D) **True.** Both are expectations over  $\mathcal{D}$ , as in Q2. This is the shared structure that makes the two terms comparable and lets them be traded off against each other.

(E) **True.** Since  $\text{Bias}^2 \geq 0$  and  $\text{Var} \geq 0$ ,

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Var} + \sigma^2 \geq \sigma^2$$

with equality only for the oracle predictor.

***Note.** Option (C) is the one worth dwelling on, because the constant predictor is the extreme point of the entire bias-variance picture: it is what ridge regression converges to as  $\lambda \rightarrow \infty$ , and what a decision stump of depth zero computes. Any time a question asks what happens “in the limit of infinite regularization”, the answer is this predictor: variance  $\rightarrow 0$ , bias  $\rightarrow$  its maximum.*

**Q4.** Expected squared test error = 0.50,  $\text{Bias}^2 = 0.20$ ,  $\sigma^2 = 0.05$ . Variance =

**Answer:** (A)

**Solution.** Rearrange the decomposition:

$$\text{Var} = \mathbb{E}[(y - \hat{f}(x))^2] - \text{Bias}^2 - \sigma^2 = 0.50 - 0.20 - 0.05 = 0.25$$

Option (B) is what you get by subtracting only  $\text{Bias}^2$  and forgetting the noise term; option (C) by subtracting only  $\sigma^2$ ; option (D) by adding rather than subtracting.

***Note.** The quantity given is already  $\text{Bias}^2$ , not bias — read the exponent. Q5 exploits the opposite reading, so the pair is best attempted together.*

- Q5.** Bias = 0.3, Variance = 0.16,  $\sigma^2 = 0.04$ . Expected squared error =

**Answer: (A)**

**Solution.** Here the *bias* is given, not the squared bias, so it must be squared before adding:

$$\text{Bias}^2 + \text{Var} + \sigma^2 = (0.3)^2 + 0.16 + 0.04 = 0.09 + 0.16 + 0.04 = 0.29$$

**The trap.** Option (B) is  $0.3 + 0.16 + 0.04 = 0.50$ , which is what you get by adding the bias without squaring it. This is the single most common arithmetic slip in this topic, and it is deliberately placed as a distractor.

***Note.** A dimensional check catches it instantly. Bias is measured in the units of  $y$ ; variance and  $\sigma^2$  are in units of  $y$  squared. Adding 0.3 to 0.16 is adding a length to an area. Only  $\text{Bias}^2$  is commensurable with the other two terms.*

- Q6.** Model A:  $\text{Bias}^2 = 0.30$ ,  $\text{Var} = 0.05$ . Model B:  $\text{Bias}^2 = 0.05$ ,  $\text{Var} = 0.35$ . Same  $\sigma^2 = 0.10$ . Which generalizes better?

**Answer: (A)**

**Solution.** Expected test error is the sum of the three terms, so compute both totals:

$$\text{Model A: } 0.30 + 0.05 + 0.10 = 0.45, \quad \text{Model B: } 0.05 + 0.35 + 0.10 = 0.50$$

Model A has the lower expected squared error and therefore generalizes better, despite being the more biased of the two.

**Why (D) is wrong.** Everything needed is given. The temptation to answer “cannot be determined” comes from an instinct that low bias is intrinsically preferable, but the decomposition is additive and both models are fully specified.

***Note.** The point of this question is that bias is not a defect to be minimized in isolation. A deliberately biased estimator with small variance routinely beats an unbiased one with large variance — this is the entire justification for ridge regression, which is biased by construction and still wins on test error. Since  $\sigma^2$  is common to both, it never affects a comparison; only the sum  $\text{Bias}^2 + \text{Var}$  matters when ranking models on the same problem.*

**Q7.** Raising the regularization strength moves variance  $0.20 \rightarrow 0.08$  and Bias<sup>2</sup> from  $0.05 \rightarrow 0.12$ , with  $\sigma^2 = 0.03$  fixed.

**Answer:** (A, B)

**Solution.** (A) **True.** Compute both totals:

$$\text{before: } 0.05 + 0.20 + 0.03 = 0.28, \quad \text{after: } 0.12 + 0.08 + 0.03 = 0.23$$

(B) **True.** The trade cost 0.07 in squared bias and bought 0.12 in variance, a net gain of 0.05. That is precisely what “worthwhile” means here.

(C) **False.** The question states that  $\sigma^2 = 0.03$  is fixed. No setting of  $\lambda$  changes the noise in the data-generating process.

(D) **False.** Nothing guarantees the next increment behaves like this one. The test-error curve as a function of  $\lambda$  is U-shaped: increasing  $\lambda$  helps while the variance reduction outweighs the bias increase, and hurts afterwards. Since we are not told where on the curve we now sit, no guarantee is available — and if  $\lambda$  keeps rising, the model eventually degenerates to the constant predictor of Q3(C).

*Note. This question is the numerical version of Q12. At the optimal  $\lambda$  the marginal changes exactly cancel, so a small step in either direction leaves total error unchanged to first order. Here the step gained 0.05, which tells you the current  $\lambda$  was still on the left of the optimum before the change — but says nothing about which side of it we are on now.*

**Q8.** Darts tightly clustered, but the cluster sits well off the bullseye:

**Answer:** (C)

**Solution.** Two independent readings of the board:

- *Where is the centre of the cluster?* Off the bullseye  $\Rightarrow$  the average prediction misses the truth  $\Rightarrow$  high bias.
- *How wide is the cluster?* Tight  $\Rightarrow$  predictions barely change across training sets  $\Rightarrow$  low variance.

High bias, low variance: option (C). This is the signature of an underfitting model — consistently wrong in the same direction.

*Note. Each dart is one model fitted to one training set, not one prediction from one model. That reading is what makes the picture a picture of the algorithm. The corresponding real-world objects are linear regression on genuinely non-linear data, a depth-1 tree, or ridge with a large  $\lambda$ .*

**Q9.** Darts scattered all over the board, but their centroid lands on the bullseye:

**Answer: (B)**

**Solution.** Apply the same two readings. The centroid is on target, so the average prediction equals the truth and bias is (near) zero. The scatter is wide, so predictions swing violently with the training sample and variance is high. Low bias, high variance: option (B).

This is the overfitting signature: the model is flexible enough to be right *on average*, but any single fit is dominated by the noise in its particular training sample.

***Note.** Averaging many such darts would land on the bullseye, and that observation is not idle — it is the whole idea behind bagging and random forests. If your errors are variance rather than bias, averaging independently-fitted models drives the error down. Averaging the darts in Q8 would leave you exactly as far off-centre as before, which is why bagging does nothing for an underfitting model.*

**Q10.** Which models plausibly produce the *scattered-but-centred* board?

**Answer: (A, B, D)**

**Solution.** Scattered-but-centred means low bias, high variance, so the question is: which of these are flexible enough to be unbiased, and unstable enough to swing with the sample?

**(A) 1-nearest neighbour — yes.** Its prediction at  $x$  is the label of a single training point, so it inherits that point's noise in full. Changing the training set changes the neighbour and hence the prediction. Nearly unbiased, notoriously high variance.

**(B) Fully grown, unpruned decision tree — yes.** It drives training error to zero by carving out leaves for individual points. A different sample produces a completely different tree structure.

**(C) Predicting the global mean of  $y$  — no.** This is the constant predictor. Variance is near zero and bias is large: the tight-but-off-centre board of Q8.

**(D) Degree-15 polynomial on 20 points — yes.** With 16 coefficients and 20 points the fit is nearly interpolating, and high-degree polynomials oscillate wildly between the data. Perturbing one point can change the curve dramatically.

**(E) Ridge with very large  $\lambda$  — no.** As  $\lambda \rightarrow \infty$  all coefficients shrink towards zero and the model tends to the constant predictor again. Same board as (C).

***Note.** (C) and (E) reaching the same place by different routes is the useful observation: heavy regularization and a deliberately trivial model class are two mechanisms for the same effect on the dartboard. Note also that (D) is really a statement about the ratio  $d/n$ , not about polynomials — a degree-15 polynomial fitted to 20,000 points is a perfectly stable, low-variance model.*

**Q11.** As model capacity increases, the typical pattern is:

**Answer: (B)**

**Solution.** Increasing capacity enlarges the hypothesis class. A larger class contains functions closer to  $f$ , so the best achievable approximation improves and **bias falls**. But a larger class also has more freedom to chase the noise in whichever sample it is given, so the fit becomes more sample-dependent and **variance rises**.

Their sum, plus the constant  $\sigma^2$ , is therefore a **U-shaped** curve: falling while the bias reduction dominates, rising once the variance increase takes over. That is option (B).

Options (A) and (D) have the direction of bias backwards. Option (C) would mean more capacity is free, contradicting everything in this section.

***Note.** Two cautions. First, the U applies to test error; training error falls monotonically and never turns up, which is what makes it useless as a capacity-selection criterion. Second, modern heavily over-parameterized networks exhibit “double descent”, where test error falls again beyond the interpolation threshold — worth knowing, but the classical U-shape is what is being tested here.*

**Q12.** The minimum of the test-error curve occurs where:

**Answer: (C)**

**Solution.** Write the expected test error as a function of capacity  $c$ :

$$E(c) = \text{Bias}^2(c) + \text{Var}(c) + \sigma^2$$

At an interior minimum the derivative vanishes:

$$E'(c) = \frac{d \text{Bias}^2}{dc} + \frac{d \text{Var}}{dc} = 0 \implies -\frac{d \text{Bias}^2}{dc} = \frac{d \text{Var}}{dc}$$

In words: the rate at which squared bias is falling exactly equals the rate at which variance is rising. That is option (C).

**Why (B) fails.** Bias equalling variance is a statement about *levels*; the optimality condition is about *slopes*. There is no reason the two curves should cross at the minimum of their sum.



**Why (A) and (D) fail.** Both name incidental landmarks. Training error hitting zero, or capacity matching  $n$ , may occur far to the right of the optimum and carry no optimality property.

*Note.* This is the general shape of every regularization argument you will meet: an optimum sits where two marginal effects cancel, never where two levels happen to coincide. The same reasoning determines the optimal  $\lambda$  in ridge, the optimal tree depth, and the optimal stopping epoch.

**Q13.** Select all true statements about the standard capacity curves.

**Answer:** (A, B, C)

**Solution. (A) True.** A larger hypothesis class contains the smaller one, so the best training fit available cannot get worse. Training error is non-increasing in capacity — weakly, since it may plateau at zero.

**(B) True.** Training error falls while test error eventually rises, so the gap between them widens. That gap is the empirical fingerprint of variance.

**(C) True.** This is Q3(E) read off the plot:  $E(c) = \text{Bias}^2 + \text{Var} + \sigma^2 \geq \sigma^2$  for every  $c$ . The noise floor is a horizontal asymptote the test curve cannot cross.

**(D) False.** Variance *increases* with capacity. This is the direction reversal that (A) and (D) of Q11 also test.

*Note.* Statement (A) is worth stating carefully: it holds for the best achievable training error within a nested family of classes. A particular optimizer on a particular run can of course report a worse training error for a bigger model — non-convexity, bad initialization, early stopping. The claim is about the hypothesis class, not the training run.

**Q14.** Train MSE = 0.02, validation MSE = 0.35, noise floor  $\approx 0.02$ . The model:

**Answer:** (B)

**Solution.** Compute the two diagnostics:

$$\text{avoidable bias} = 0.02 - 0.02 = 0.00, \quad \text{variance gap} = 0.35 - 0.02 = 0.33$$

The training error has already reached the noise floor, so there is no bias left to remove — the model fits the training data as well as anything possibly could. Meanwhile the validation error is more than an order of magnitude larger. All the error is in the gap. This is textbook overfitting: high variance, option (B).

**Why (A) fails.** Underfitting would show a *large* training error. Here it is at the floor.

**Why (D) fails.** The noise floor is given as 0.02, which is small; the 0.35 cannot be blamed on irreducible error.

*Note. Sensible next moves all attack variance: more training data, a stronger penalty, fewer features, early stopping, bagging. Note what is not on the list — adding capacity or features, which would make the gap worse.*

**Q15.** Train error = 0.24, validation error = 0.26, noise floor  $\approx 0.02$ . Best next step:

**Answer: (C)**

**Solution.** The two diagnostics:

$$\text{avoidable bias} = 0.24 - 0.02 = 0.22, \quad \text{variance gap} = 0.26 - 0.24 = 0.02$$

The gap is negligible, so variance is not the problem. The training error itself sits far above the noise floor, so the model is not even fitting the data it has been shown. That is underfitting, and the fix is more capacity: option (C).

**Why (A) fails — and this is the important one.** More data reduces variance. Variance here is already about 0.02. Even driving it to zero would leave you at roughly 0.24. This is exactly the situation of Q20: two flat, converged, closely-spaced learning curves sitting well above the target, where additional data buys essentially nothing.

**Why (B) and (D) fail.** Both are variance-reduction moves. Applied to an underfitting model they increase bias and make things worse.

*Note. The practical lesson is that “collect more data” is the reflexive answer and is wrong roughly half the time. Reading the gap first tells you whether data will help before you spend anything acquiring it.*

**Q16.** Train error = 0.20, validation error = 0.45, noise floor  $\approx 0.02$ .

**Answer: (A, B, C)**

**Solution.**

$$\text{avoidable bias} = 0.20 - 0.02 = 0.18, \quad \text{variance gap} = 0.45 - 0.20 = 0.25$$

**(A) True and (B) True,** by the two computations above.

**(C) True.** Both quantities are large. The model is simultaneously failing to fit its training data and failing to generalize what it does fit. Nothing forbids this; the two failure modes are independent, and it is the common situation early in a project.

**(D) False.** Shrinking the model attacks only the 0.25 and would worsen the 0.18. A single-lever fix cannot address both terms; you need a bias move and a variance move — for instance a richer model class together with more data or a tuned penalty.

*Note.* The three-way split of the validation error is the useful summary:

$$0.45 = \underbrace{0.02}_{\text{noise floor}} + \underbrace{0.18}_{\text{avoidable bias}} + \underbrace{0.25}_{\text{variance gap}}$$

*Writing it out this way tells you not just what is wrong but how much of the total each problem is worth, which is how you decide what to work on first.*

**Q17.** With the hypothesis class held fixed, adding more training data:

**Answer: (A)**

**Solution.** With the class fixed, the best approximation to  $f$  available inside that class does not change — so the systematic component of the error, the bias, is essentially unchanged. What changes is stability: a larger sample averages out more of the noise, so the fitted model moves less from one sample to the next and **variance falls**, tending to zero as  $n \rightarrow \infty$  for a consistent estimator. That is option (A).

**Why (B) fails.** It has the two effects exactly swapped. Data cannot fix a systematically wrong hypothesis class: fitting a line to a parabola with a million points still gives a line.

**Why (C) fails.**  $\sigma^2$  is a property of the distribution, as established in Q3(B).

**Why (D) fails.** Variance is about the spread of the *estimator*, which shrinks with  $n$ ; it is not the total quantity of noise in the dataset.

*Note.* The qualifier “with the hypothesis class held fixed” is doing real work. In practice more data also lets you afford a richer class, which then lowers bias too — but that is a second decision, not an automatic consequence. On the learning-curve picture: more data pulls the validation curve down towards the training curve, and both converge to the best error the class can achieve. If that asymptote is above your target, no quantity of data will reach it.

**Q18.** You add many extra features, some informative and some pure noise.

**Answer: (A, B, D)**

**Solution. (A) True.** The informative features enlarge what the model can express, so the best achievable approximation improves and bias tends to fall.

**(B) True.** Every additional feature is another parameter estimated from the same finite sample. The noise features carry no signal but still receive fitted coefficients, which chase sampling noise and make the fit less stable.

**(C) False, in the sense intended.** With the target's conditional distribution fixed,  $\sigma^2$  is what it is. A genuinely new measurement that explains previously unexplained variation in  $y$  *would* lower the noise floor — but that redefines the problem, and the option as written asserts a general consequence of adding features, which does not hold.

**(D) True.** As  $d$  approaches  $n$  the model has nearly as many parameters as observations, and the fit approaches interpolation of the training set. This is the regime of Q10(D).

*Note. This question is the practical face of Q11. “More features” is a capacity increase, and it obeys the same trade: bias down, variance up, with an optimum somewhere in between. It is also the motivation for the entire Regularization section — if you cannot resist adding features, you can at least penalize their coefficients.*

**Q19.** Which of the following statements are True?

**Answer: (A, D)**

**Solution. (A) True.** More data reduces variance (Q17). If the diagnosed problem is variance, more data is exactly the right lever, and its effect is reliable rather than incidental.

**(B) False — read the option carefully.** Curves that *converge* to a common value have a small gap, and the gap is the fingerprint of variance. Converging to a *high* value means both errors are large with little separating them, which is high *bias*. This is the Q15 situation. The option correctly describes the picture and then attaches the wrong label to it.

**(C) False.** Irreducible error is not a property of the hypothesis class (Q3(B), Q1).

**(D) True.** Low training error with a much higher cross-validation error is the definition of the variance gap, which is the Q14 situation.

*Note. (B) and (D) between them are the whole learning-curve diagnostic: it is the gap that reports variance and the level that reports bias. Curves close together and low: healthy. Close together and high: bias. Far apart: variance. Far apart and high: both.*

**Q20.** Learning curves have flattened, are close to each other, and both sit well above the desired performance level. Adding more training data will most likely:

**Answer: (B)**

**Solution.** Three features of the description each independently point the same way. *Flattened* means the curves have stopped responding to  $n$ , so the asymptotic regime has been reached. *Close together* means the variance gap is small, and variance is the only thing data reduces. *Well above the target* means the remaining error is bias.

Adding data therefore acts on a term that is already near zero, and leaves untouched the term that dominates. Option (B): little effect.

**Why (A) fails.** It would require a large variance gap for the data to close.

**Why (C) fails.** Training error does typically rise slightly with  $n$  as the model can no longer memorize, but the accompanying sharp fall in test error is exactly what a converged, closely-spaced pair of curves rules out.

**Why (D) fails.** Q3(B) again.

*Note. This is Q15 presented as a picture rather than as three numbers, and it is worth being able to move between the two representations fluently. The actionable version: before commissioning a data-collection effort, plot the learning curve on the data you already have. If the two curves have already met, the money is better spent on model capacity or on better features.*

**Q21.** Regularized logistic regression: 2% training error, 22% cross-validation error. Which steps are likely to help?

**Answer: (A, C)**

**Solution.**

$$\text{variance gap} = 22\% - 2\% = 20\%$$

A large gap with a very low training error is the overfitting signature of Q14. Every sensible move must reduce variance.

**(A) Increase  $\lambda$  — helps.** A stronger penalty shrinks the coefficients, reduces the effective capacity, and trades a little bias for a large reduction in variance.

**(B) Decrease  $\lambda$  — hurts.** This increases capacity and widens the gap. It is the correct move for the *opposite* diagnosis.

**(C) Get more training examples — helps.** The canonical variance fix, and here the gap is large enough for it to have room to work.

**(D) Add more polynomial features — hurts.** A capacity increase (Q18), applied to a model that already has too much of it.

***Note.** Note the pairing: (A)/(B) and (C)/(D) are opposite-signed levers, and the diagnosis picks one from each pair. If the numbers had instead read 24% training and 26% validation, the correct selections would flip to (B) and (D). Reading the gap first, then choosing levers, is more reliable than memorizing which actions are “good”.*

**Q22.** Increasing the polynomial degree from 2 to 12 with the training set size fixed.

**Answer:** (A, B, D)

**Solution.** Raising the degree is a capacity increase, so Q11 supplies the answers directly.

**(A) True.** Degree-2 polynomials are a subset of degree-12 polynomials, so the best achievable training fit cannot get worse (Q13(A)).

**(B) True.** The richer class approximates  $f$  better, so bias falls.

**(C) False.** Variance *increases*. Twelve coefficients estimated from the same fixed sample are less stable than three, and high-degree polynomials oscillate badly between the data points.

**(D) True.** This is the U-shape. If degree 2 was underfitting, test error falls at first as bias is removed, then turns and rises once the growing variance dominates.

***Note.** “Keeping the training set size fixed” is the load-bearing clause. The variance increase in (C) reflects the ratio of parameters to observations, so the same jump from degree 2 to degree 12 accompanied by a tenfold increase in  $n$  could reduce test error monotonically with no visible U at all. Capacity is only ever excessive relative to the amount of data.*

## 2 Validation

### Key ideas for this section

1. **Three sets, three jobs, and they do not substitute for one another.** The training set fits parameters. The validation set chooses between fitted models — hyperparameters, feature sets, algorithms. The test set is opened once, at the end, to report a number. The moment you look at the test set and change something in response, it has become a validation set and its unbiasedness is gone. Almost half the questions in this section are a restatement of this one rule.
2.  **$k$ -fold mechanics.** Partition the training data into  $k$  equal folds. Each round trains on  $k - 1$  folds and scores on the one held out, so every point is validated exactly once and used for training  $k - 1$  times. One hyperparameter setting therefore costs  $k$  fits, and a grid of  $m$  settings costs  $mk$ . LOOCV is the special case  $k = N$ .
3. **The bias and variance of the *estimate*, not of the model.** This is the subtlety that Q7 and Q8 turn on. Small  $k$  means each model trains on a small fraction of the data, so it is worse than the model you will eventually deploy and the error estimate is *pessimistically biased*. Large  $k$  means each model trains on nearly all the data — low bias — but the  $k$  training sets are then almost identical, the  $k$  scores are strongly correlated, and averaging correlated quantities does little to reduce variance. So bias falls and variance rises as  $k$  grows, and  $k = 5$  or  $10$  is the usual compromise.  
  

$$\text{small } k \rightarrow \text{pessimistic bias,} \quad k \rightarrow N \rightarrow \text{low bias, high variance}$$
4. **Cost arithmetic.** Holdout: 1 fit.  $k$ -fold:  $k$  fits per setting. Grid of  $m$  settings:  $mk$ . Nested CV with  $K$  outer and  $L$  inner folds:  $K \cdot L \cdot m$ . LOOCV:  $N$ . The only escape is a closed-form shortcut such as the PRESS identity for linear smoothers, which is algorithm-specific and not available in general.
5. **Split along the right unit, and preserve the right proportions.** Leakage is not only about the target variable. If the natural unit is a patient, a customer or a time period, then splitting rows at random puts correlated records on both sides and inflates the score — the fix is to change the *splitting scheme* (group  $k$ -fold, forward chaining), never to change  $k$ . For imbalanced classification, stratification keeps each fold's class ratio equal to the population's, which prevents folds with no positives and lowers the variance of the estimate.

**Q1.** Holdout on a very small dataset: the most significant statistical risk.

**Answer:** (B)

**Solution.** With a single split, your reported number is computed on one particular subset. Change the random seed and you get a different subset,



and on a small dataset a different answer — possibly a very different one. That instability across possible splits is variance in the estimator.

**Why (A) fails.** Training never touches the test set, so the model cannot overfit to it. Overfitting to a test set happens only through repeated human-in-the-loop selection (Q19(B)), which is a different mechanism and is not automatic.

**Why (C) fails.** A single holdout split is the *cheapest* scheme available: one fit. Cost is the reason people reach for holdout, not a risk of it.

**Why (D) fails.** Nothing about a small dataset causes leakage. Leakage comes from computing something on the whole dataset before splitting, or from splitting along the wrong unit (Q14) — both of which are procedural mistakes, not consequences of size.

***Note.** The size of the test set, not the size of the dataset, drives this. A binomial standard error argument makes it concrete: an accuracy estimated on  $n$  test points has standard error roughly  $\sqrt{p(1-p)/n}$ , which for  $p \approx 0.9$  and  $n = 100$  is about 3%. Two models three points apart are then statistically indistinguishable. This is exactly the problem  $k$ -fold is designed to solve.*

**Q2.** Statements about train/validation/test split ratios.

**Answer: (B, C)**

**Solution. (A) False.** There is no universally optimal ratio, and 50/50 is a poor default: it wastes half the data on evaluation. The right ratio depends on how many points the evaluation needs to be stable, which is an absolute count, not a proportion.

**(B) True, and it is the key insight about ratios.** What matters is the *number* of evaluation rows, not their share. One percent of ten million rows is 100,000 rows — vastly more than enough for a tight estimate. Giving the test set 20% here would mean two million rows, which buys nothing and costs the model nearly two million training examples.

**(C) True.** This is Q1 restated: a small test set gives a high-variance estimate.

**(D) False.** Roles reversed. Final unbiased reporting is the test set's job; the validation set has been used repeatedly for selection and is therefore optimistically biased (Q13, Q19).

***Note.** (B) and (C) are the two ends of the same principle and are worth holding together: allocate as few rows to evaluation as will make the estimate stable, and give the rest to training. On a thousand-row dataset that*



*reasoning pushes you towards cross-validation instead, because no fixed split is simultaneously large enough to evaluate on and small enough to spare.*

**Q3.** Number of model fits required by LOOCV on  $N$  observations.

**Answer: (B)**

**Solution.** LOOCV is  $k$ -fold with  $k = N$ : each fold contains exactly one observation, so there are  $N$  rounds, and each round refits the model from scratch on the other  $N - 1$  points. That is  $N$  fits.

Option (A) confuses the number of *fits* with the size of each *training set*, which is indeed  $N - 1$ . Option (C) would arise if each round required  $N$  fits, which it does not. Option (D) is the answer for general  $k$ -fold, of which this is the  $k = N$  case.

**Note.** Two qualifications for later. First, for linear smoothers the PRESS identity

$$CV_{(N)} = \frac{1}{N} \sum_{i=1}^N \left( \frac{e_i}{1 - h_{ii}} \right)^2$$

*delivers the LOOCV error from a single fit on the full data, using the residuals  $e_i$  and the leverages  $h_{ii}$ . That is an algorithm-specific shortcut, not a contradiction of this answer. Second, the phrase “from scratch” in the question is what makes (B) unambiguous.*

**Q4.** True characteristics of LOOCV.

**Answer: (A, C)**

**Solution. (A) True.** There is no random partitioning to do. The folds are forced: fold  $i$  is observation  $i$ . Run LOOCV twice and you get exactly the same number, whereas 5-fold with a different seed gives a different one. This is a genuine and often overlooked advantage.

**(B) False.** Precisely backwards.  $N$  full training runs of a deep network on a large dataset is the most expensive validation scheme imaginable.

**(C) True.** Any two LOOCV training sets share  $N - 2$  of their  $N - 1$  points. The resulting models are nearly identical, so their errors are strongly correlated — and averaging  $N$  strongly correlated quantities reduces variance far less than averaging  $N$  independent ones. This is the mechanism behind LOOCV’s high variance.

**(D) False.** LOOCV trains on  $N - 1$  points, essentially the full dataset, so the model being evaluated is almost exactly the model you will deploy. That makes the estimate nearly unbiased — *low* bias. Its weakness is variance, not bias.

***Note.** (C) and (D) together are the whole argument for preferring  $k = 10$  to  $k = N$ : you give up a little bias, in exchange for  $k$  far less correlated fold scores and a much cheaper computation. Note that (A) and (C) are not in tension — determinism is a statement about the partition, while variance here is about the spread of the estimate across hypothetical redraws of the dataset itself.*

- Q5.** Grid search over 4 settings with 5-fold cross-validation: total models trained.

**Answer:** (C)

**Solution.** Cross-validation is run *once per hyperparameter setting*, and each run costs  $k$  fits:

$$4 \text{ settings} \times 5 \text{ folds} = 20 \text{ models}$$

Option (A) counts one round of CV only, forgetting that the grid has four points. Option (B) is  $4 + 5$ , adding what should be multiplied. Option (D) is  $5^2$ .

***Note.** Add the final refit and the count becomes 21: after the grid identifies a winner, the deployed model is refitted on the full training data with those hyperparameters (Q17). Nested cross-validation multiplies by yet another factor: with  $K$  outer folds it becomes  $K \cdot L \cdot m$  fits before any refit.*

- Q6.** What happens during standard  $k$ -fold cross-validation.

**Answer:** (A, B, D)

**Solution.** (A) **True** and (B) **True**. These are the definition. Each round holds out one fold and trains on the other  $k - 1$ .

(C) **False**. The fold models are diagnostic instruments, not deliverables. Picking the best-scoring one is doubly wrong: it was trained on only  $k - 1$  folds, so it saw less data than necessary; and selecting the *maximum* of  $k$  noisy scores is selection on noise, so that fold's advantage is partly luck and will not transfer. Standard practice is the refit of Q17.

(D) **True**. The folds partition the data, so each point lies in exactly one held-out fold and therefore contributes to the validation score exactly once. This is what makes the  $k$  fold scores averageable into a single estimate covering all  $N$  points.

***Note.** Statement (D) is the property that distinguishes  $k$ -fold from repeated random splits (also called Monte Carlo CV or shuffle-split), where subsets are drawn independently each round: there a point can appear in several validation sets or in none. The partition property is why  $k$ -fold is the default.*

**Q7.** LOOCV ( $k = N$ ) compared with 5-fold, as an estimator of test error.

**Answer:** (B)

**Solution.** Consider the two properties separately, remembering that both refer to the *estimate of error*, not to the model.

**Bias.** LOOCV trains each model on  $N - 1$  points; 5-fold trains each on  $0.8N$ . Since learning curves are decreasing, a model trained on 80% of the data is genuinely worse than one trained on nearly all of it, so 5-fold's estimate is pessimistically biased — it reports the error of a slightly weaker model than you will deploy. LOOCV's models are essentially the deployed model, so its bias is lower.

**Variance.** By Q4(C), the  $N$  LOOCV models are nearly identical and their errors are strongly correlated. For an average of  $n$  quantities with pairwise correlation  $\rho$ ,

$$\text{Var}\left(\frac{1}{n} \sum_i Z_i\right) = \frac{\sigma_Z^2}{n} [1 + (n-1)\rho] / n \cdot n = \rho \sigma_Z^2 + \frac{(1-\rho)\sigma_Z^2}{n}$$

which is bounded below by  $\rho \sigma_Z^2$  no matter how large  $n$  becomes. High  $\rho$  therefore defeats the benefit of averaging many folds. LOOCV has the higher variance.

Lower bias and higher variance: option (B).

***Note.** The bias-variance trade-off appears twice in this course and the two appearances must not be confused. In Section 1 it describes a predictor. Here it describes an estimator of a predictor's error. Choosing  $k$  is a trade-off in the second sense, which is why the recommendation ( $k = 5$  or  $10$ ) sits in the interior rather than at either extreme.*

**Q8.** Why 5- or 10-fold is preferred over 2-fold.

**Answer:** (B)

**Solution.** In 2-fold, each model trains on half the data. Reading off a typical learning curve, a model trained on  $N/2$  points is materially worse than one trained on  $N$ , so the reported error overstates the error of the model you will actually ship. That is pessimistic bias, and it is the dominant objection to  $k = 2$ .

**Why (A) fails.** 2-fold requires *two* fits, fewer than the five or ten of the alternatives. Cost argues *for*  $k = 2$ , not against it.

**Why (C) fails.** The folds are disjoint by construction, so there is no contamination. Contamination arises from preprocessing before splitting or from the wrong split unit, independent of  $k$ .

**Why (D) fails.** Cross-validation replaces the *validation* set, not the test set. A held-out test set remains advisable at any  $k$  (Q19(C)).

*Note. Note the direction of the bias, which students routinely get backwards. Too-small training folds make cross-validation pessimistic; reusing the folds for selection makes the reported best score optimistic (Q13). Both effects are present in a normal workflow, they push in opposite directions, and neither cancels the other reliably.*

**Q9.** 4-fold cross-validation on parts  $f_1, f_2, f_3, f_4$ .

**Answer: (B)**

**Solution.** With  $k = 4$ , each round trains on  $k - 1 = 3$  folds and validates on the 1 fold left out, and this is repeated four times so that each part serves as the validation fold once.

**Why (A) fails.** It inverts the proportions: one fold for training and three for validation. That would train each model on 25% of the data.

**Why (C) fails.** A 2/2 split is 2-fold cross-validation applied to four labelled parts, not 4-fold.

**Why (D) fails — and this is the interesting distractor.** The split proportions are right, but the terminology is not. The held-out fold is a *validation* fold, used inside the model-selection loop and then rotated. A test set is a separate body of data that is not rotated, not reused, and not touched until the end. Calling the rotating fold a test set collapses precisely the distinction that Q19 is about.

*Note. The general statement:  $k$ -fold trains on a fraction  $(k - 1)/k$  and validates on  $1/k$ , for  $k$  rounds. For  $k = 4$  that is 75% and 25%. Options (A) and (B) differ only in which number is which, so reading the fractions rather than pattern-matching the sentence is the reliable approach.*

**Q10.** How a cross-validated ridge penalty  $\lambda > 0$  helps, relative to unregularized least squares.

**Answer: (A)**

**Solution.** For OLS, the covariance of the estimator is  $\sigma^2(X^\top X)^{-1}$ , so the total variance of the coefficients is

$$\sum_j \text{Var}(\hat{\beta}_j) = \sigma^2 \text{tr}((X^\top X)^{-1}) = \sigma^2 \sum_j \frac{1}{d_j^2}$$

where  $d_j^2$  are the eigenvalues of  $X^\top X$ . When the design is nearly collinear, some  $d_j^2$  are close to zero, the reciprocals blow up, and the estimates are wildly unstable.

Ridge replaces  $X^T X$  by  $X^T X + \lambda I$ , whose eigenvalues are  $d_j^2 + \lambda$ . Every term of the trace shrinks:

$$\frac{1}{d_j^2 + \lambda} < \frac{1}{d_j^2} \quad \text{for every } \lambda > 0$$

and the smallest eigenvalues — the ones causing the trouble — are helped proportionally the most. Lower total variance is exactly what buys the reduction in expected test error.

**Why (B) fails.** It has the inequality the wrong way round.

**Why (C) fails.** Ridge *introduces* bias; that is the price paid. OLS is the unbiased estimator here.

**Why (D) fails.**  $\sigma^2$  is a property of the data-generating process and no estimator touches it.

*Note. Read carefully, cross-validation is not what shrinks the trace — the penalty does that, for any  $\lambda > 0$ . Cross-validation's role is to pick which  $\lambda$ , i.e. how much variance reduction is worth how much bias. That division of labour is the link between this section and the next.*

**Q11.** Selecting the regularization parameter  $\lambda$ .

**Answer: (B)**

**Solution.**  $\lambda$  is a hyperparameter, and hyperparameters are chosen on the validation set or by cross-validation. That is the definition of what those sets are for.

**Why (A) fails.** Training error is monotonically increasing in  $\lambda$ , so minimizing it always returns  $\lambda = 0$  and the criterion is vacuous. Any criterion that cannot ever select a positive penalty is not a criterion for selecting a penalty.

**Why (C) fails.** Tuning on the test set is precisely the practice Q19(B) prohibits. It would give the best-looking number and the least trustworthy one.

**Why (D) fails.** Large  $n$  reduces the *need* for regularization, since variance falls with sample size, but it does not make the optimal  $\lambda$  exactly zero, and it says nothing about the  $p > n$  or near-collinear cases where the penalty is doing structural work.

*Note. Worth pairing with the one-standard-error rule: rather than the  $\lambda$  at the literal CV minimum, take the largest  $\lambda$  whose CV error is within one standard error of it. The minimum is itself a noisy quantity, and this rule deliberately errs towards the simpler model.*

**Q12.** Which of the following statements are True?

**Answer:** (A, B, D)

**Solution. (A) True.** The model was optimized on the training data, so it fits the idiosyncrasies of that particular sample as well as the signal. Training error is therefore an optimistically biased estimate of test error. In degenerate cases the two can coincide, hence “typically”.

**(B) True, and it is quantifiable.** In 5-fold, two different training sets each consist of four of the five folds, and they share three:  $3/5 = 60\%$  of the data is common. The resulting models are similar, and their scores correlated — the mechanism of Q4(C) and Q7, and the fact Q20(C) uses.

**(C) False.** That is the validation set’s job. Using the test set for tuning destroys it.

**(D) True.**  $\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Var} + \sigma^2 \geq \sigma^2$ , carried over from Section 1.

***Note.** Statement (B) is the quantitative core of this section. It explains why cross-validation scores are correlated, why LOOCV’s variance is high, why  $k$ -fold cannot be treated as  $k$  independent experiments, and why a paired statistical test across folds needs care — the standard  $t$ -test’s independence assumption is violated by exactly this overlap.*

**Q13.** Why the best cross-validation score is an optimistically biased estimate of generalization performance.

**Answer:** (B)

**Solution.** Suppose you evaluate  $m$  hyperparameter settings on the same folds. Each setting’s CV score is its true performance plus fold-specific noise. Taking the *maximum* over  $m$  such scores selects partly for genuine quality and partly for a favourable noise draw — the winner is the configuration that happened to suit these particular folds. Reporting that maximum as an unbiased estimate therefore overstates performance, and the bias grows with  $m$ .

**Why (A) fails.** Increasing the number of folds changes the variance and the bias of each individual estimate, but does nothing about the selection effect. Even with  $k = 50$ , taking the best of many would still be optimistic.

**Why (C) fails.** Cross-validation without selection is close to unbiased, or mildly *pessimistic* for small  $k$  (Q8). The optimism here comes entirely from the tuning step, not from cross-validation itself.

**Why (D) fails.** Stratification addresses fold composition for imbalanced classes (Q15). It is unrelated to the multiple-comparisons effect described here.

***Note.** Two standard remedies. Keep a genuinely untouched test set and report on that; or use nested cross-validation, where the inner loop selects and the outer loop scores, so that no data is used both to choose and to judge. The cost is  $K \cdot L \cdot m$  fits.*

**Q14.** A dataset with multiple records per patient.

**Answer:** (A, B, C)

**Solution. (A) True.** Records from one patient are correlated — shared physiology, shared history, sometimes near-duplicate rows. If some of a patient’s records train the model and others validate it, the model can recognise the patient rather than learn the condition, and the score is inflated.

**(B) True.** Grouping by patient ID keeps every record of a patient on the same side of the split. The resulting estimate answers the question you actually care about — how does this model perform on a patient it has never seen?

**(C) True, and this is the conceptual point.** Leakage is usually taught as “information about the target crossing the split”, for instance scaling or target-encoding before splitting. Here nothing was computed from  $y$  at all; the leak is structural, caused by splitting along the wrong unit. Both are leakage.

**(D) False.** Increasing  $k$  changes how many folds there are, not what a row means. Every fold under a row-wise scheme still splits patients across the boundary. The fix must change the splitting scheme itself.

***Note.** The general principle: split along the unit at which you want to generalize. Predicting for new patients means splitting by patient; predicting the future means splitting by time (forward chaining, never random shuffling); predicting for new stores means splitting by store. A row-wise random split silently assumes rows are exchangeable, which is a modelling assumption worth stating rather than inheriting from a default argument.*

**Q15.** Why stratified  $k$ -fold is preferred when the positive class is 2%.

**Answer:** (B)

**Solution.** Random folds receive roughly 2% positives — but only roughly. With  $N = 500$  and  $k = 5$ , each fold of 100 points expects 2 positives, and the actual count fluctuates between 0 and 6 or so. A fold with zero positives makes recall undefined and precision meaningless, and even folds with one or two positives give scores that jump discontinuously with a single classification. Averaging such folds gives a very noisy estimate.



Stratification allocates positives and negatives to folds separately, so each fold holds the population ratio by construction. This removes the empty-fold failure and sharply reduces the variance of the averaged score.

**Why (A) fails.** The number of models is  $k$  either way; stratification changes fold *composition*, not fold count.

**Why (C) fails.** Nothing about an evaluation protocol can guarantee anything about overfitting. Stratification changes how you *measure*, not what you fit.

**Why (D) fails.** Q19(C): the test set survives every change of validation scheme.

***Note.** Stratification is cheap and essentially never harmful, so it is the sensible default for classification rather than a special-case tool. It also composes with the grouping of Q14 — stratified group  $k$ -fold keeps patients intact and balances class ratios across folds, which is what a clinical dataset usually needs.*

**Q16.** The purpose of repeated  $k$ -fold cross-validation.

**Answer: (B)**

**Solution.** A single run of  $k$ -fold depends on one random partition. A different seed produces different folds and a different score; that seed-to-seed spread is a real source of variance in the estimate. Repeating the whole procedure  $r$  times with fresh partitions and averaging the  $r \times k$  results averages that partition noise away.

**Why (A) fails.** Bias comes from the size of the training folds, which is  $(k - 1)/k$  of the data in every repeat. Repeating does not change it.

**Why (C) fails.** Nothing changes  $\sigma^2$ .

**Why (D) fails.** Repeats concern the folds, which live entirely inside the training data. The test set is untouched and its single-use rule is unaffected.

***Note.** Note the boundary with LOOCV. Repeated  $k$ -fold reduces the variance attributable to which partition you drew, and it does so effectively because different repeats give genuinely different partitions. It does not reduce the variance attributable to which dataset you drew, which is the irreducible part of any validation estimate. LOOCV, by contrast, has no partition randomness at all ( $Q4(A)$ ) — so repeating it would be pointless.*

**Q17.** Producing the final deployed model after  $k$ -fold selection.

**Answer: (C)**

**Solution.** Cross-validation's output is a *hyperparameter setting*, together with an estimate of how a model built that way performs. The  $k$  fold models were instruments used to obtain that estimate, and each saw only  $(k - 1)/k$  of the data. Once the setting is chosen, refit on the full training data: more data means lower variance, so the refitted model is at least as good as any fold model, and the CV estimate you already have is a fair — if slightly pessimistic — forecast of its performance.

**Why (A) and (D) fail.** Both deploy a fold model, which was trained on less data than available. (A) compounds the error by selecting on the maximum of  $k$  noisy scores, which is the selection effect of Q13 in miniature.

**Why (B) fails.** Averaging the  $k$  fold models is a legitimate technique and sometimes performs well — it is a form of bagging. But “as the only option” overstates it: it is one choice among several, it costs  $k$  times as much at inference, and the standard practice remains the single refit.

*Note. This is where the pessimistic bias of  $k$ -fold becomes a feature rather than a defect. Your reported CV score describes a model trained on  $(k - 1)/k$  of the data; what you ship is trained on all of it, and so should be slightly better. Erring in that direction is the right way round.*

**Q18.** Statements about the cost of validation schemes.

**Answer:** (A, B, C)

**Solution.** (A) **True.** One split, one training run.

(B) **True.**  $k$  folds means  $k$  fits, and the count multiplies across a hyperparameter grid, as in Q5.

(C) **True.**  $k = N = 10,000$  gives 10,000 fits. For anything more expensive than a small linear model this is out of reach — the practical reason LOOCV is confined to tiny datasets.

(D) **False, and the stated reasoning is the trap.** It is true that consecutive LOOCV training sets differ by one point. But that observation would only reduce cost if the fit could be *updated* incrementally rather than recomputed, which is possible for a narrow class of models and not in general. For a neural network, “differs by one point” still means a full retraining run. Counting fits: 10,000 versus 10.

*Note. The exception proves the rule. For linear smoothers, the PRESS identity of Q3's remark computes the entire LOOCV error from one fit — so for those models LOOCV genuinely is cheaper than 10-fold. Option (D) is wrong because it asserts this always holds and attributes it to the wrong reason.*

**Q19.** The relationship between the validation set and the test set.

**Answer:** (A, B, C)

**Solution.** (A) **True.** This is the division of labour: validation selects, test reports, once.

(B) **True, and it is the rule that is hardest to keep.** Nothing technical stops you from scoring on the test set repeatedly; the damage is statistical and invisible. Each look followed by an adjustment is one more selection step, and after enough of them the test set is functionally a validation set with an optimistically biased score. No re-labelling of the file restores its unbiasedness.

(C) **True.** Cross-validation replaces the validation *set*, but the selection effect of Q13 applies to CV scores just as it does to a fixed validation set. An untouched test set is what remains uncontaminated by selection.

(D) **False.** Both scores are estimates with variance, and the validation score has been inflated by selection. A gap between them is normal; a guarantee is not available.

*Note. This is the reasoning behind competition design and behind reproducibility practice more broadly: a held-out set whose labels the modeller cannot see cannot be optimized against. The organizational version of the same idea is to write the test set to a separate location and open it once, at the end, with the number reported whatever it turns out to be.*

**Q20.** When to use  $n$ -fold cross-validation and when to use a separate test set.

**Answer:** (C)

**Solution.** The two halves of (C) are separate arguments and both are sound.

**Small dataset  $\Rightarrow$  cross-validation.** Setting aside a test set removes those rows from the learning set **L**, which is already scarce. Cross-validation recycles: every point trains in  $k - 1$  rounds and validates in one, so no data is permanently withheld.

**Large dataset  $\Rightarrow$  separate test set.** Two things change. You can afford to withhold rows, since a small *fraction* still gives a large *count* (Q2(B)). And cross-validation's benefit shrinks: the fold learning sets overlap heavily — in 5-fold, two training sets share three of five folds, so 60% of the data is common — the models are therefore very similar, and averaging their highly correlated scores adds much less information than  $k$  independent evaluations would. Paying  $k$  times the compute for that is poor value when a single large test set already gives a tight estimate.

**Why (A) fails.** It reverses both recommendations. It also misdiagnoses the constraint: the difficulty on a small dataset is that a test set starves the learning set, not that folds lack examples.

**Why (B) fails.** Averaging over folds reduces variance, not bias, and only partially, because the fold scores are correlated. Small  $k$  in fact *introduces* pessimistic bias (Q8).

**Why (D) fails.** It inverts the computational argument. Cross-validation costs  $k$  fits and holdout costs one, so it is holdout that saves computation — and on large data, where fits are most expensive, that is exactly where holdout is used.

***Note.** The 60% figure generalizes to  $(k - 2)/k$ : for two distinct  $k$ -fold training sets, the shared fraction is  $3/5$  at  $k = 5$ ,  $8/10$  at  $k = 10$ , and  $(N - 2)/(N - 1)$  under LOOCV — which is  $Q_4(C)$  again, at the far end of the same scale. One number, read at three values of  $k$ , explains the entire section: it is why fold scores are correlated, why LOOCV's variance is high, why 5 or 10 is the compromise, and why on large data a single held-out set is enough.*

**VisionGATE**  
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

**VisionGATE**  
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

### 3 Regularization

#### Key ideas for this section

1. **Two formulations of the same problem.** Every penalty has a budget form and a Lagrangian form, and they trace out the same solution path:

$$\min_{\beta} \|y - X\beta\|_2^2 + \lambda \Omega(\beta) \iff \min_{\beta} \|y - X\beta\|_2^2 \text{ subject to } \Omega(\beta) \leq t$$

with  $t$  decreasing as  $\lambda$  increases: a heavier penalty is a tighter budget. The penalized form is what you optimize; the constrained form is what you draw. Every geometric argument in this section — circles, diamonds, corners — lives in the second form.

2. **Ridge fixes invertibility by moving the spectrum.** Differentiating the ridge objective gives  $(X^\top X + \lambda I)\beta = X^\top y$ . If  $X^\top X$  has eigenvalues  $d_j^2 \geq 0$ , then  $X^\top X + \lambda I$  has eigenvalues  $d_j^2 + \lambda$ , every one of which is strictly positive for  $\lambda > 0$ . The matrix is therefore positive definite, hence invertible, hence the solution is unique — *regardless* of collinearity or of  $p > N$ . This single fact answers Q1, Q2 and Q14.
3. **Sparsity is a corner, not a magnitude.**  $\|\beta\|_1 \leq t$  is a cross-polytope whose vertices lie on the coordinate axes;  $\|\beta\|_2 \leq t$  is a smooth ball with no distinguished points. A smooth contour first touching a smooth ball almost never does so at a coordinate axis, whereas a vertex catches a whole range of contour orientations — so lasso produces exact zeros and ridge does not. The analytic counterpart is that  $|\beta|$  is *not* differentiable at 0 while  $\beta^2$  is.
4. **The penalty is not invariant, so the parametrization matters.** Rescaling a feature rescales its coefficient inversely, which leaves OLS fitted values unchanged but changes how hard that coefficient is penalized. Hence: standardize the features first, using training-set statistics. And leave the intercept unpenalized, so that adding a constant to  $y$  shifts  $\hat{\beta}_0$  and nothing else.

**Q1.** The closed-form ridge solution.

**Answer:** (B)

**Solution.** Differentiate the objective  $\|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2$ :

$$\nabla_{\beta} = -2X^\top y + 2X^\top X\beta + 2\lambda\beta = 0 \implies (X^\top X + \lambda I)\hat{\beta} = X^\top y$$

$$\text{so } \hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y.$$

**Why the others fail.** (A) is the OLS solution, the  $\lambda = 0$  case. (C) *subtracts*  $\lambda I$ , which pushes eigenvalues towards zero and makes the conditioning worse rather than better — exactly backwards. (D) omits the inverse entirely and is not even dimensionally a solution of the normal equations.

***Note.** The  $\lambda\|\beta\|_2^2$  term contributes  $2\lambda\beta$  to the gradient, which is what adds  $\lambda I$  to the matrix. That is the whole derivation, and it is worth being able to reproduce in two lines: nearly everything else in this section follows from the resulting eigenvalue shift  $d_j^2 \mapsto d_j^2 + \lambda$ .*

**Q2.** Why ridge guarantees a unique solution when  $p > N$  or under perfect collinearity.

**Answer:** (B, C)

**Solution.**  $X^\top X$  is symmetric positive *semi*-definite: its eigenvalues  $d_j^2$  are non-negative but may be zero. They are zero exactly when  $X$  has linearly dependent columns, which is automatic when  $p > N$  and also occurs under perfect collinearity. A matrix with a zero eigenvalue is singular, so OLS has no unique solution.

Adding  $\lambda I$  shifts every eigenvalue up by  $\lambda$ :

$$\text{eig}(X^\top X + \lambda I) = \{d_j^2 + \lambda\} \geq \lambda > 0$$

All eigenvalues are strictly positive, so the matrix is positive definite and therefore invertible, and the ridge objective is strictly convex with exactly one minimizer.

**(B) and (C) are two statements of this same fact** — positive definite *means* all eigenvalues positive — so both are true.

**Why (A) fails.** Ridge does not drop anything. Every coefficient stays in the model, merely shrunk (Q16). Feature elimination is lasso's behaviour, not ridge's.

**Why (D) fails.** The claim inverts the situation. Squared-error loss is convex and  $\lambda\|\beta\|_2^2$  is strictly convex, so their sum is *strictly convex*. Convexity is precisely why the minimum is unique; a non-convex problem would give no such guarantee.

***Note.** Distinguish the two regimes this question merges. Under perfect collinearity or  $p > N$ ,  $X^\top X$  is genuinely singular and ridge makes the solution exist uniquely. Under near collinearity it is invertible but ill-conditioned, and ridge makes the solution stable by keeping  $1/(d_j^2 + \lambda)$  bounded where  $1/d_j^2$  would explode. Q14 is the first regime; the variance argument of Section 2's Q10 is the second.*

**Q3.** The geometric property of the  $L_1$  penalty that produces sparsity.

**Answer:** (C)

**Solution.** The constraint region  $\|\beta\|_1 \leq t$  is a diamond in 2D, a cross-polytope in general, and its vertices sit on the coordinate axes — points where all but one coordinate is exactly zero. The solution is where the

smallest reachable loss contour first touches this region. A vertex is a point of many supporting hyperplanes, so it catches contours arriving from a whole range of directions, and a touch at a vertex means a coefficient is exactly zero.

**Why (A) fails, and it is the informative distractor.**  $|\beta|$  is emphatically *not* differentiable at the origin: its derivative jumps from  $-1$  to  $+1$ . That kink is the analytic form of the geometric corner, and it is the *cause* of sparsity, not a property lasso lacks. A differentiable penalty like  $\beta^2$ , whose derivative vanishes at zero, exerts no force on an already-small coefficient and so never drives it to exactly zero.

**Why (B) fails.** Squaring is the  $L_2$  penalty.  $L_1$  takes absolute values.

**Why (D) fails.** Lasso applies soft thresholding, which moves coefficients *towards* zero and clips them there; it never pushes a positive coefficient past zero into negative territory.

*Note. The word “intersect” in the option is loose. A contour that genuinely crossed into the constraint region could not be optimal — you could slide inwards to a strictly lower contour still meeting the region. The correct relation is touching: the smallest contour that meets the region at all. At a smooth boundary point that touching is tangency; at a vertex there is no tangent line, which is precisely why a corner can be the answer for many different contour orientations.*

**Q4.** Statements about the 2D geometry.

**Answer: (A, B, D)**

**Solution. (A) True.**  $\beta_1^2 + \beta_2^2 \leq t$  is a disc, and a ball in higher dimensions.

**(B) True.**  $|\beta_1| + |\beta_2| \leq t$  is a diamond with vertices at  $(\pm t, 0)$  and  $(0, \pm t)$ ; in  $p$  dimensions it is the cross-polytope.

**(C) False.**  $\hat{\beta}_{OLS}$  is the centre of the contour family, determined entirely by the data. Where it falls relative to a constraint region depends on  $t$ , and there is no reason for it to land exactly on a boundary — that would be a measure-zero coincidence.

**(D) True, and it is the case people forget to draw.** If  $\hat{\beta}_{OLS}$  already satisfies the constraint, it minimizes the loss over the whole space and *a fortiori* over the feasible set. The constraint is inactive, the KKT multiplier is zero, and the penalized solution equals the OLS solution.

*Note. (D) is why the standard figure always shows the contour centre outside the region: the inside case is real but visually trivial. In the Lagrangian form the same statement reads that the interesting range of  $\lambda$  is*



where the constraint binds. Note carefully that this does not make “ $\hat{\beta}_{OLS}$  inside the region” the normal case — cross-validation typically selects a  $t$  strictly smaller than  $\|\hat{\beta}_{OLS}\|$  even for a well-behaved model, because trading a little bias for a larger drop in variance pays.

**Q5.** Ridge as  $\lambda \rightarrow \infty$ .

**Answer: (D)**

**Solution.** From the shrinkage factor  $d_j^2/(d_j^2 + \lambda)$ , every component tends to zero as  $\lambda \rightarrow \infty$ , so  $\hat{\beta} \rightarrow 0$  and the model degenerates to predicting the intercept — the mean of  $y$  — for every input.

That predictor has no dependence on the training sample beyond  $\bar{y}$ , so its **variance tends to zero**. It is also as wrong as the model class permits, so its **bias is at its maximum**. Coefficients  $\rightarrow 0$ , bias maximal, variance  $\rightarrow 0$ : option (D).

**Why (A) fails.** Nothing grows; the penalty punishes magnitude.

**Why (B) fails.** That describes  $\lambda \rightarrow 0$ , the other endpoint (Q19).

**Why (C) fails.** It gets the coefficients right and both error terms exactly backwards.

***Note.** The limit is the constant predictor from Section 1 — the tight, off-centre dartboard. That the two extremes of  $\lambda$  land on OLS at one end and the global mean at the other is the cleanest way to remember which direction bias and variance move, since both endpoints are models you can picture without any algebra.*

**Q6.** Ridge and lasso under collinearity.

**Answer: (A, B)**

**Solution.** Take two features that are near-duplicates. The loss barely distinguishes between giving weight  $c$  to one,  $c$  to the other, or splitting it — only the sum is well determined. The penalty breaks the tie, and the two penalties break it differently.

**(A) True.** For a fixed sum  $\beta_1 + \beta_2 = c$  with both positive,  $\|\beta\|_1 = c$  is *constant*: the  $L_1$  penalty is indifferent to how the weight is divided. With nothing to prefer a split, the optimum sits at a vertex of that flat region, and lasso picks one feature essentially arbitrarily — a different sample can flip which one, which is why lasso’s selection is unstable under collinearity.

**(B) True.**  $\|\beta\|_2^2$  is strictly convex, and for a fixed sum  $c$  it is minimized at  $\beta_1 = \beta_2 = c/2$ . Ridge therefore actively prefers an even split and pulls correlated coefficients towards each other.

(C) **False.** Both shrink; neither preserves the OLS values.

(D) **False.** This is the exact opposite of Q2. Perfect collinearity is what breaks OLS and what ridge was designed to survive.

*Note. The instability in (A) is the practical objection to using lasso for interpretation on correlated data: the story “the model selected feature 7” may not survive a resample. Elastic net (Q13) exists to fix precisely this, and its grouping effect is inherited from the  $L_2$  term’s preference described in (B).*

**Q7.** Standardizing features and not penalizing the intercept.

**Answer: (A, B, D)**

**Solution. (A) True.** The intercept absorbs the location of  $y$ . If you replace  $y$  by  $y + c$ , the sensible response is  $\hat{\beta}_0 \mapsto \hat{\beta}_0 + c$  with all slopes unchanged. Penalizing  $\beta_0$  shrinks it towards zero, so the fitted model would depend on where you happened to put the origin of the target — an arbitrary choice of units, not a property of the data.

(B) **True.** The penalty treats all coefficients by the same rule, so that rule is only fair if the coefficients are comparable. Standardizing puts every feature on a common scale, so that coefficient magnitude reflects predictive contribution rather than choice of unit.

(C) **False.** It asserts the opposite of (A), and the stated mechanism is invented.

(D) **True — check the direction carefully.** A distance in kilometres carries *smaller* numbers than the same distance in millimetres. For the fitted values to agree,  $\beta_{\text{km}} = 10^6 \times \beta_{\text{mm}}$ . So a feature with a small numerical range does carry a large coefficient, and since the penalty punishes large coefficients, that feature is shrunk harder purely because of its units.

*Note. Option (D) is easy to key wrongly, so it is worth the ten seconds: write  $x_{\text{mm}} = 10^6 x_{\text{km}}$ , note that  $\beta x$  must be invariant, and read off which coefficient is the big one. Q15 makes the same point without the units.*

**Q8.** The limitation of lasso that elastic net primarily addresses.

**Answer: (B)**

**Solution.** When  $p > n$ , the lasso solution can have at most  $n$  non-zero coefficients — a consequence of the KKT conditions and the geometry of the  $L_1$  ball, and a hard ceiling regardless of how much signal is present. In settings like genomics, where  $p$  is tens of thousands and  $n$  is a few hundred, that ceiling is a real limitation. Adding an  $L_2$  term removes it: elastic net can select more than  $n$  variables.

**Why (A) fails.** Categorical variables are handled by encoding, at the feature-construction stage, and are unrelated to either penalty.

**Why (C) fails.** Lasso has no closed form and ridge does, so lasso is indeed the costlier of the two; but coordinate descent and LARS make it entirely practical, and elastic net does not make it cheaper.

**Why (D) fails.** Selection is the one thing lasso does that ridge cannot.

***Note.** The other standard motivation is the grouping effect (Q13(B)): the  $L_2$  term restores ridge's preference for splitting weight evenly among correlated predictors, curing the arbitrary-selection instability of Q6(A). The question says "primarily" and offers the saturation ceiling, so (B) is the intended answer, but in practice the grouping effect is the more commonly cited reason.*

**Q9.** Increasing  $\lambda$  in ridge regression.

**Answer:** (A)

**Solution.** A larger  $\lambda$  shrinks the coefficients further from their unbiased OLS values, so the estimator becomes more biased. It also makes the fit less responsive to the particular training sample — in the limit, completely unresponsive (Q5) — so variance falls. Bias up, variance down.

***Note.** This is Q5 stated as a direction rather than a limit, and the two should be answered consistently. The reason the trade is worth making at all is that near the optimum the variance reduction outweighs the bias increase; that is the numerical content of Section 1's Q7.*

**Q10.** Very large  $\lambda$ , high error on both training and test sets.

**Answer:** (A, B)

**Solution.** High error on *both* sets means a small train–test gap and a high level: the diagnosis is high bias, not high variance. Section 1's rule applies, and every correct move must increase capacity.

**(A) Helps.** A very large  $\lambda$  has over-shrunk the model towards the constant predictor. Reducing it restores flexibility.

**(B) Helps.** Polynomial features enlarge the hypothesis class, which is the direct attack on bias.

**(C) Hurts.** Less data never lowers test error. It raises variance while leaving the bias problem untouched.

**(D) Hurts.** Removing features reduces capacity further, in a model that already has too little.

***Note.** Note that (A)/(B) and (C)/(D) are not arbitrary distractors: (A) and (D) are opposite-signed levers, as are the capacity moves. Had the numbers instead shown low training error and high test error, the correct selections would flip to (C) and (D). Reading the gap first, then choosing levers, is more reliable than memorizing which actions are “good”.*

**Q11.** Penalized form versus constrained form.

**Answer:** (B)

**Solution.** The constrained problem has Lagrangian

$$\mathcal{L}(\beta, \lambda) = \|y - X\beta\|_2^2 + \lambda (\|\beta\|_2^2 - t)$$

and since the problem is convex with a strictly feasible point, strong duality holds. Minimizing the Lagrangian in  $\beta$  is exactly the penalized problem, the  $-\lambda t$  term being constant in  $\beta$ . So for each  $\lambda \geq 0$  there is a  $t$  — namely  $t = \|\hat{\beta}_\lambda\|_2^2$  — giving the same solution, and the two formulations trace the same path.

The direction matters: a *larger*  $\lambda$  penalizes magnitude more heavily and yields a *smaller* solution norm, so  $t$  decreases as  $\lambda$  increases. The end-points agree with this:  $\lambda = 0$  corresponds to  $t = \|\hat{\beta}_{\text{OLS}}\|_2^2$  (no effective constraint), and  $\lambda \rightarrow \infty$  to  $t \rightarrow 0$ .

**Why (D) fails.** It has the monotonicity backwards, which would make  $\lambda \rightarrow \infty$  correspond to an unbounded budget.

**Why (C) fails.** Orthonormality gives *closed forms* (uniform rescaling for ridge, soft thresholding for lasso) but is irrelevant to the equivalence, which follows from convex duality alone.

***Note.** The correspondence is data-dependent: the  $t$  matching a given  $\lambda$  depends on  $X$  and  $y$ , so you cannot convert between them without solving the problem. This is why software exposes  $\lambda$  — it is the quantity you can set in advance — while textbooks draw  $t$ , since the budget is what has a picture.*

**Q12.** The lasso regularization path as  $\lambda$  decreases towards zero.

**Answer:** (A)

**Solution.** At  $\lambda$  large enough, every coefficient is zero. As  $\lambda$  falls, the budget loosens and predictors enter the active set one at a time — at the value of  $\lambda$  where a variable’s correlation with the current residual first justifies a non-zero coefficient. The path is piecewise linear in  $\lambda$ , with kinks exactly at these entry (and occasional exit) events, and the number of non-zero coefficients broadly increases until at  $\lambda = 0$  the solution is OLS.

**Why (B) fails.** Coefficients activate at different thresholds; a single common threshold is exactly what does *not* happen.

**Why (C) fails.** Direction reversed: sparsity decreases as the penalty weakens.

**Why (D) fails.** The coefficients vary continuously along the path; nothing jumps at the end.

***Note.** “Generally increases” is doing careful work in the option. The active set is not strictly monotone — a variable that entered early can leave again as others enter and absorb its explanatory role. The LARS algorithm exploits the piecewise-linear structure to compute the entire path in roughly the cost of a single OLS fit, which is why plotting coefficients against  $\lambda$  is cheap and standard practice.*

**Q13.** True statements about the elastic net.

**Answer:** (A, B, C)

**Solution. (A) True.** Immediate from the objective  $\|y - X\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$ : killing either term leaves the other penalty alone.

**(B) True.** The strictly convex  $L_2$  term prefers to split weight evenly among correlated predictors (Q6(B)), so they tend to enter and leave together with similar coefficients — the grouping effect, and the cure for lasso’s arbitrary picking.

**(C) True.** The  $L_1$  term keeps the corners of the constraint region, so exact zeros remain possible. The elastic net ball is a rounded diamond: still pointed on the axes, but bulged elsewhere.

**(D) False.** Removing the  $n$ -variable ceiling is the whole point of Q8. Elastic net can select more than  $n$  variables when  $p > n$ .

***Note.** Options (C) and (D) between them locate elastic net precisely: it keeps what lasso has that ridge lacks (sparsity) and discards what lasso has that ridge lacks (the saturation limit and the unstable selection). The cost is a second hyperparameter, so the cross-validation grid becomes two-dimensional.*

**Q14.** The OLS problem when  $p > N$ .

**Answer:** (A)

**Solution.** With more columns than rows,  $\text{rank}(X) \leq N < p$ , so  $X^\top X$  is  $p \times p$  with rank at most  $N$  and is therefore singular. The normal equations  $X^\top X\beta = X^\top y$  have infinitely many solutions: any element of the null space of  $X$  can be added to a solution without changing  $X\beta$ .

Geometrically, the columns of  $X$  span the whole of  $\mathbb{R}^N$ , so  $y$  can be fitted exactly — infinitely many  $\beta$  achieve zero training error, and least squares has no way to prefer one. A penalty supplies the missing criterion and selects a unique, stable answer.

**Why (B) fails.** That describes near-collinearity with  $p \leq N$ , where the matrix is invertible but poorly conditioned. Here it is exactly singular.

**Why (C) fails.** There is no unique solution, and the estimator's variance is unbounded rather than zero.

**Why (D) fails.** It reverses the geometry:  $y$  can be projected onto the column space perfectly, which is the problem, not an impossibility.

*Note.* Zero training error with infinitely many minimizers is a useful thing to hold onto: it shows that “fits the training data perfectly” carries no information at all in this regime. The choice among the perfect fits is made entirely by the penalty, and different penalties choose differently —  $L_2$  picks the minimum-norm solution,  $L_1$  picks a sparse one.

**Q15.** Statements about scaling.

**Answer:** (A, B, C)

**Solution. (A) True.** Replace  $x_j$  by  $cx_j$ . The OLS fit is unchanged with  $\hat{\beta}_j \mapsto \hat{\beta}_j/c$ , since  $(\hat{\beta}_j/c)(cx_j) = \hat{\beta}_j x_j$ . Least squares depends only on the column space of  $X$ , which rescaling does not alter.

**(B) True.** The penalty sees  $\hat{\beta}_j/c$ , not  $\hat{\beta}_j$ . Halving a feature's units doubles its coefficient and quadruples its  $L_2$  contribution, so the same model is penalized differently under a different parametrization. Neither ridge nor lasso is scale-invariant.

**(C) True, with the important qualifier.** Standardization is a fitted transformation: the means and standard deviations are parameters estimated from data. Computing them on the full dataset before splitting leaks test information into training, so they must come from the training folds only and then be *applied* to validation and test data.

**(D) False.** It asserts the negation of (B). The functional form of the penalty is exactly why scaling matters, not why it does not.

*Note.* (A) and (B) together explain why standardization is optional for OLS and mandatory for penalized regression: the invariance that makes OLS indifferent is precisely what the penalty destroys. (C) is the operational half, and it is the point of contact with Section 2 — “fit the scaler on training data only” is the same discipline as “do not compute anything on the full dataset before splitting”.

**Q16.** Ridge regression for finite  $\lambda > 0$ .

**Answer: (A)**

**Solution.** From the shrinkage factor  $d_j^2/(d_j^2 + \lambda)$ : for any finite  $\lambda$  this is strictly between 0 and 1, so components are reduced but never annihilated. The analytic reason is that the penalty's derivative  $2\lambda\beta_j$  vanishes as  $\beta_j \rightarrow 0$  — the force pushing a coefficient towards zero dies away as it gets close, so it approaches zero asymptotically without arriving. Lasso's derivative is  $\pm\lambda$ , a constant force that does not weaken, which is why it produces exact zeros.

**Why (B) fails.** That is lasso's behaviour (Q3).

**Why (C) fails.** The slopes are precisely what shrinks; the intercept is normally left unpenalized (Q7).

**Why (D) fails.** Direction reversed.

*Note.* This is the single most useful contrast in the section, and it can be checked in one line for an orthonormal design, where ridge gives  $\hat{\beta}_j^{ols}/(1 + \lambda)$  — a uniform rescaling that never hits zero — and lasso gives  $\text{sign}(\hat{\beta}_j^{ols})(|\hat{\beta}_j^{ols}| - \lambda/2)_+$ , the soft-thresholding operator, whose positive part clips to exactly zero.

**Q17.**  $w = [4, 6, 8, 10]$  at  $\lambda = 3$ ;  $\lambda$  raised to 6.

**Answer: (B)**

**Solution.** A larger penalty shrinks coefficients towards zero, so the updated vector must be smaller in magnitude, component by component. Only  $[1, 3, 6, 9]$  is uniformly smaller than  $[4, 6, 8, 10]$ .

Options (A) and (C) are both larger, contradicting the direction of shrinkage. Option (D) claims no change, but the solution depends continuously and strictly monotonically on  $\lambda$ .

*Note.* Notice the shrinkage is not uniform: the four components are scaled by 0.25, 0.5, 0.75 and 0.9. That is not a defect of the question but a feature of ridge on a general design — by the SVD result, components lying along poorly determined directions are shrunk hardest while well-determined ones are barely touched. Only for an orthonormal  $X$  would every coefficient be divided by the same  $1 + \lambda$ .

**Q18.** Lasso on a dataset with a single feature,  $\lambda$  raised to a high value.

**Answer: (A)**

**Solution.** With one feature the model is  $\hat{y} = \beta_0 + \beta_1 x$ , and the lasso solution for  $\beta_1$  is a soft-thresholded version of the least-squares slope.



Once  $\lambda$  exceeds the threshold,  $\beta_1$  is set to exactly zero and the fitted model reduces to  $\hat{y} = \bar{y}$ : a horizontal line at the mean of the target.

**Why (B) fails.** A vertical line is not the graph of a function of  $x$  and cannot be produced by this model at any  $\lambda$ .

**Why (C) fails.** The slope shrinks continuously with  $\lambda$  well before it reaches zero, so the line changes throughout.

**Why (D) fails.** Direction reversed: the penalty reduces  $|\beta_1|$ .

***Note.** Because the intercept is unpenalized (Q7(A)), the line settles at  $\bar{y}$  rather than at zero — it flattens without falling to the  $x$ -axis. Ridge in the same setting would drive the slope towards zero asymptotically, approaching a horizontal line but never becoming exactly horizontal at any finite  $\lambda$ ; that difference is Q16 in its smallest possible instance.*

**Q19.** Ridge as  $\lambda \rightarrow 0$ .

**Answer:** (C)

**Solution.** Setting  $\lambda = 0$  in  $\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$  returns  $(X^\top X)^{-1} X^\top y$ , the ordinary least-squares estimator; equivalently, every shrinkage factor  $d_j^2/(d_j^2 + \lambda) \rightarrow 1$ . The penalty term vanishes from the objective and the problem is plain least squares.

Options (A) and (E) describe magnitudes rather than a limiting procedure, and (A) is the  $\lambda \rightarrow \infty$  limit in any case (Q5). Option (B) has no meaning here. Option (D) confuses the two penalties: ridge does not become lasso at any  $\lambda$ ; they are different penalties, and both reduce to OLS at  $\lambda = 0$ .

***Note.** Together with Q5, this pins down both endpoints of the ridge path — OLS at one end, the constant predictor at the other — and every question about the direction of bias, variance or coefficient magnitude can be answered by asking which endpoint you are moving towards. Note that the limit requires  $X^\top X$  to be invertible: when  $p > N$  (Q14), the  $\lambda \rightarrow 0$  limit of ridge is the minimum-norm least-squares solution rather than “the” OLS solution, since no unique OLS solution exists.*